ISLAMIC UNIVERSITY OF TECHNOLOGY

DHAKA, BANGLADESH

ORGANIZATION OF ISLAMIC COOPERATION

DATABASE MANAGEMENT SYSTEMS LAB

Name: Antara Arifa Mullick

Student ID: 210042132

Department: Computer Science and Engineering

Program: B.Sc. in Software Engineering

Report preparing for Lab Task

Restaurant Simulation Documentation

Introduction

The objective of this project is to implement a multithreaded restaurant simulation in Java. The simulation involves customers placing orders, chefs preparing food, waiters serving dishes, and a receptionist managing table assignments.

Customer Class

- The `Customer` class represents individual customers in the restaurant.

- Each customer is assigned a unique identifier (`customerId`).

- Random orders are generated for each customer, consisting of a combination of dishes (Burger, Pizza, Pasta).

- The `placeOrder()` method is responsible for placing a random order for each customer.

- The `Receptionist` class is utilized to assign a table to each customer using `Thread.join()`.

Chef Class

- The `Chef` class simulates chefs preparing food in the restaurant kitchen.

- Each chef is assigned a unique identifier (`chefId`).

- The `run()` method continuously checks the order queue (`orderQueue`) for new orders.

- If an order is present, the chef simulates cooking time and adds the cooked food to the cooked food queue (`cookedFoodQueue`).

Waiter Class

- The `Waiter` class simulates waiters serving dishes to customers.

- Each waiter has a unique identifier (`waiterId`).

- The `run()` method continuously checks the cooked food queue (`cookedFoodQueue`) for dishes to serve.

- If cooked food is present, the waiter simulates serving time and uses `Thread.join()` to synchronize with the chef thread before serving the dish.

Receptionist Class

- The `Receptionist` class manages table assignments for customers.

- It keeps track of available tables and assigns a table to each customer.

- The `assignTable()` method is synchronized to ensure proper table assignment.

- The `run()` method continuously checks for waiting customers and assigns tables using `Thread.join()`.

RestaurantSimulation Class

- The `RestaurantSimulation` class is the main class responsible for creating and managing threads for chefs, waiters, and customers.

- Shared data structures (`orderQueue` and `cookedFoodQueue`) are used to represent the order queue and cooked food queue, respectively.

- The `Receptionist` class is created to manage table assignments for customers.

- Threads are created for chefs, waiters, and customers and added to the `threads` list.

- All threads are started to simulate concurrent activities in the restaurant.

- The simulation continues until all customers are served, and `Thread.join()` is used to ensure the correct termination of threads at the end of the simulation.

Challenges Faced

1. Synchronization Issues:

- Ensuring proper synchronization was essential to avoid race conditions.

2. Data Integrity:

- Shared data structures (`orderQueue` and `cookedFoodQueue`) required careful handling to prevent data corruption.

3. Implementation of Thread.join():

- `Thread.join()` was used in the `Receptionist` and `Waiter` classes to synchronize the termination of customer threads.

4. Randomization:

- `Thread.sleep()` was employed to simulate processing time for orders, cooking, serving, etc.